

Dart Programming

Lesson 7:

Creating Scrolling Lists and Effects

USING THE CARD

- The **Card** widget is part of the Material Design and has minimal rounded corners and shadows.
- To group and lay out data, the Card is a perfect widget to enhance the look of the UI.
- The **Card** widget is customizable with properties such as **elevation**, **shape**, **color**, **margin**, and others.
- The **elevation** property is a value of double, and the higher the number, the larger the shadow that is cast.
- To customize the shape and borders of the **Card** widget, we modify the shape property.
- Some of the shape properties are **StadiumBorder**, **UnderlineInputBorder**,
- **OutlineInputBorder**, and others.

USING THE CARD

- Card(
 - elevation: 8.0,
 - color: Colors.white,
 - margin: EdgeInsets.all(16.0),
 - child: Column(
 - mainAxisAlignment: MainAxisAlignment.center,
 - children: <Widget>[
 - Text(
 - 'Vacation Plans',
 - textAlign: TextAlign.center,
 - style: TextStyle(
 - fontWeight: FontWeight.bold,
 - fontSize: 48.0,
 - color: Colors.orange,)),
 -),
 - Text(
 - 'Travel Plans',
 - textAlign: TextAlign.center,
 - style: TextStyle(color: Colors.grey)),),
 -],),

USING THE CARD

- The following are a few ways to customize the Card's shape property
- // Create a Stadium Border
 - shape: StadiumBorder(),
- // Create Square Corners Card with a Single Orange Bottom Border
 - shape: UnderlineInputBorder(borderSide: BorderSide(color: Colors.deepOrange)),
- // Create Rounded Corners Card with Orange Border
 - shape: OutlineInputBorder(borderSide: BorderSide(color: Colors.deepOrange.
 - withOpacity(0.5))),

USING THE LISTVIEW AND LISTTILE

- The constructor **ListView.builder** is used to create an on-demand linear scrollable list of widgets
- When you have a large set of data, the **builder** is called only for visible widgets, which is great for performance.
- Within the builder, you use the **itemBuilder** callback to create the list of children widgets.
- Keep in mind the **itemBuilder** is called only if the **itemCount** argument is greater than zero, and it is called as many times as the **itemCount** value.
- Remember, the List starts at row 0, not 1. If you have 20 items in the List, it loops from row 0 to 19.
- The **scrollDirection** argument defaults to **Axis.vertical** but can be changed to **Axis.horizontal**.

USING THE GRIDVIEW

- The **GridView** displays tiles of scrollable widgets in a grid format.
- The three constructors are **GridView.count**, **GridView.extent**, and **GridView.builder**.
- The **GridView.count** and **GridView.extent** are usually used with a fixed or smaller data set.
- Using these constructors means that all of the data, not just visible widgets, is loaded at **init**.
- If you have a large set of data, the user does not see the **GridView** until all data is loaded, which is not a great user experience (UX).
- Usually you use the **GridView.count** when you need a layout with a fixed number of tiles in the crossaxis.
- You use the **GridView.extent** when you need a layout with the tiles that need a maximum cross-axis extent.
- The **GridView.builder** constructor is used with a larger, infinite, or unknown size set of data.
- Like the **ListView.builder**, when you have a large set of data, the builder is called only for visible widgets, which is great for performance.

Using the GridView.count

- The **GridView.count** requires setting the **crossAxisCount** and the children argument.
- The **crossAxisCount** sets the number of tiles to display and children is a list of widgets.
- The **scrollDirection** argument sets the main axis direction for the Grid to scroll, either **Axis.vertical** or **Axis.horizontal**, and the default is vertical.
- For the children, you use a list of values. Within the children argument.
 - GridView.count(
 - crossAxisCount: 3,
 - padding: EdgeInsets.all(8.0),
 - children: List.generate(7000, (index) { *// just a sample list, replace it with the real data list*
 - print('_buildGridView \$index');
 - return Card(
 - margin: EdgeInsets.all(8.0),
 - child: InkWell(
 - child: Column(
 - mainAxisAlignment: MainAxisAlignment.center, children: <Widget>[

Using the GridView.count

- Icon(
 - _iconList[0],
 - size: 48.0,
 - color: Colors.blue,),
 - Divider(),
 - Text(
 - 'Index \$index',
 - textAlign: TextAlign.center,
 - style: TextStyle(
 - fontSize: 16.0,),)
 -],
 -),
 - onTap: () {
 - print('Row \$index');
 - },
 -),
 -);
 - },),)

Using the GridView.extent

- The **GridView.extent** requires you to set the **maxCrossAxisExtent** and **children** argument.
- The **maxCrossAxisExtent** argument sets the maximum size of each tile for the axis.
- The **scrollDirection** argument sets the main axis direction for the grid to scroll, either **Axis.vertical** or **Axis.horizontal**, and the default is vertical.
 - GridView.extent(
 - maxCrossAxisExtent: 175.0,
 - scrollDirection: Axis.horizontal,
 - padding: EdgeInsets.all(8.0),
 - children: List.generate(20, (index) {*// just a sample list, replace it with the real data list*
 - print('_buildGridViewExtent \$index');
 - return Card(
 - margin: EdgeInsets.all(8.0),
 - child: InkWell(
 - child: Column(
 - mainAxisAlignment: MainAxisAlignment.center, children: <Widget>[

Using the GridView.extent

- Icon(
 - _iconList[index],
 - size: 48.0,
 - color: Colors.blue,),
- Divider(),
- Text(
 - 'Index \$index',
 - textAlign: TextAlign.center,
 - style: TextStyle(
 - fontSize: 16.0,),
-)
-],
-),
- onTap: () {
- print('Row \$index');
- },
-),
-);
- }},

Using the GridView.builder

- The **GridView.builder** requires you to set the **itemCount**, **gridDelegate**, and **itemBuilder** arguments.
- The **itemCount** sets the number of tiles to build.
- The **gridDelegate** is a **SilverGridDelegate** responsible for laying out the children list of widgets for the **GridView**.
- The **gridDelegate** argument cannot be null; you need to pass the **maxCrossAxisExtent** size.
 - For example, to display three tiles across the screen you specify the **gridDelegate** argument with the **SilverGridDelegateWithFixedCrossAxisCount** class to create a grid layout with a fixed number of tiles for the cross axis.
 - If you need to display tiles that have a maximum width of 150.0 pixels, you specify the **gridDelegate** argument with the **SilverGridDelegateWithMaxCrossAxisExtent** class to create a grid layout with tiles that have a maximum cross-axis extent, the maximum width of each tile.

Creating the GridView.builder App

- Create a new Flutter project.
- For this project, you need to create only the pages, classes, and widgets folders.
- Create the Home Class as a **StatelessWidget** since the data does not require changes.
- Open the **home.dart** file and add to the body a **SafeArea** with the **GridViewBuilderWidget()** widget class as a child.
 - body: SafeArea(
 - child: const GridViewBuildWidget(),
 -),
- Add to the top of the file the import statement for the **gridview_builder.dart** widget class that you'll create next.
 - import 'package:flutter/material.dart';
 - import 'package:/widgets/gridview_builder.dart';
- Create a new Dart file in the widgets folder. Right-click the classes folder and then select New ➡ Dart File, enter **grid_icons.dart**, and click the OK button to save.
- Import the **material.dart** library, add a new line, and create the **GridIcons** Class.
- The **GridIcons** Class holds a List of **IconData** called **iconList**.

Creating the GridView.builder App

- Create the **getIconList()** method that creates the List of **IconData** that is used later in the **GridView.builder**.
 - class GridIcons {
 - List<IconData> iconList = [];
 - List<IconData> getIconList() {
 - iconList
 - ..add(Icons.free_breakfast)
 - ..add(Icons.access_alarms)
 - ..add(Icons.directions_car)
 - ..add(Icons.flight)
 - ..add(Icons.cake)
 - ..add(Icons.card_giftcard)
 - ..add(Icons.change_history)
 - ..add(Icons.face)
 - ..add(Icons.star)
 - ..add(Icons.headset_mic)
 - ..add(Icons.lightbulb_outline);
 - return iconList;
 - }
 - }

Creating the GridView.builder App

- Create a new Dart file under the widgets folder. Right-click the widgets folder and then select New ⇨ Dart File, enter **gridview_builder.dart**, and click the OK button to save.
- Import the **material.dart** library, add a new line, and create a **StatelessWidget** and give it a name of **GridViewBuilderWidget**.
- Modify the **GridViewBuilderWidget** widget class to return a **GridView.builder** with the **itemCount** argument set to 20 for example.
- For the **gridDelegate** argument, use **SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent: 150.0)**.
- For the **itemBuilder** callback, it passes the **BuildContext** and the widget index as an int value.
- In the **itemBuilder** first line, place a print statement to show each item index being built according to the visible space.
- Return a Card with the child as an **InkWell**.
- The **InkWell onTap** has a print statement to show the tapped Card item, with the Row selected.

Creating the GridView.builder App

- For the **InkWell** child property, pass a Column with these children: **Icon**, **Divider**, and **Text** widgets.
- Note that the **itemBuilder** is called for each row item.
- For every row, a Widget is created and added to the widget tree.
- The **onTap** will print the index of the tapped Grid item.
- Add to the top of the file the import statement for the **grid_icons.dart** class.
 - `import 'package:flutter/material.dart';`
 - `import 'package:ch9_gridview/classes/grid_icons.dart';`
 - `class GridViewBuilderWidget extends StatelessWidget {`
 - `const GridViewBuilderWidget({`
 - `Key key,`
 - `}) : super(key: key);`
 - `@override`
 - `Widget build(BuildContext context) {`
 - `List<IconData> _iconList = GridIcons().getIconList();`
 - `return GridView.builder(`
 - `itemCount: 20,`
 - `padding: EdgeInsets.all(8.0),`

Creating the GridView.builder App

- `gridDelegate:`
`SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent:`
- `150.0),`
- `itemBuilder: (BuildContext context, int index) {`
- `print('_buildGridViewBuilder $index');`
- `return Card(`
- `color: Colors.lightGreen.shade50,`
- `margin: EdgeInsets.all(8.0),`
- `child: InkWell(`
- `child: Column(`
- `mainAxisAlignment: MainAxisAlignment.center,`
- `children: <Widget>[`
- `Icon(`
- `_iconList[index],`
- `size: 48.0,`
- `color: Colors.lightGreen,`
- `),`

Creating the GridView.builder App

- Divider(),
- Text('Index \$index',
- textAlign: TextAlign.center,
- style: TextStyle(
- fontSize: 16.0,
-),
-)
-],
-),
- onTap: () {
- print('Row \$index');
- },
-),
-);
- },
-);
- }
- }

USING THE STACK

- The **Stack** widget is commonly used to overlap, position, and align widgets to create a custom look.
- The **Stack** children list of the widget is either **positioned** or **nonpositioned**.
- When you use a Positioned widget, each child widget is placed at the appropriate location.
- The **Stack** widget resizes itself to accommodate all of the **nonpositioned** children.
- The **nonpositioned** children are positioned to the alignment property (Top-Left or Top-Right depending on the left-to-right or right-to-left environment)
- Each Stack child widget is drawn in order from bottom to top.
- This means the first widget drawn is at the bottom of the Stack, and then the next widget is drawn above the previous widget and so on.
- Each child widget is positioned on top of each other in the order of the Stack children list. The **RenderStack** class handles the stack layout.

USING THE STACK

- To align each child in the Stack, you use the Positioned widget.
- By using the top, bottom, left, and right properties, you align each child widget within the Stack.
- The height and width properties of the Positioned widget can also be set.
- You implement the **FractionalTranslation** class to position a widget fractionally
- outside the parent widget.
- You set the **translation** property with the **Offset(dx, dy) (double type value for x-and y-axis)** class that's scaled to the child's size, resulting in moving and positioning the widget. For example, to show a favorite icon moved a third of the way to the upper right of the parent widget, you set the **translation** property with the **Offset(0.3, -0.3)** value.

USING THE STACK

- The following example shows a Stack widget with a background image, and by using the **FractionalTranslation** class, you set the **translation** property to the **Offset(0.3, -0.3)** value, placing the star icon one-third to the right of the x-axis and a negative one-third (move icon upward) on the y-axis.
 - Stack(
 - children: <Widget>[
 - Image(image: AssetImage('assets/images/dawn.jpg')),
 - Positioned(
 - top: 0.0,
 - right: 0.0,
 - child: FractionalTranslation(
 - translation: Offset(0.3, -0.3),
 - child: CircleAvatar(
 - child: Icon(Icons.star),
 -),
 -),
 -),
 - Positioned(/* Eagle Image */),
 - Positioned(/* Bald Eagle */),
 -],
 -),